

Temperature-Controlled Fan System

Introduction to Electronics Projects

University of Nevada, Las Vegas

Tsukasa Takahashi, Zihan Chen,
Jeremiah Dacanay, Mason Cucuru

April 27, 2026

1. Objective

The purpose of this project was to design and test a temperature-controlled fan system using an Arduino. The system was built so that the fan speed changes automatically based on the surrounding temperature. Instead of using a manual switch, the Arduino reads the temperature from the sensor and sends a PWM signal to control the speed of the DC motor.

2. Project Description

Our project uses a DHT11 temperature sensor, an Arduino Uno, a transistor, a diode, and a DC motor. The temperature sensor measures the surrounding air temperature and sends the reading to the Arduino. The Arduino then uses the code to convert the temperature reading into a fan speed value. When the temperature rises, the fan speed increases. When the temperature is lower, the fan speed decreases.

3. Components Used

The main components used in this project were an Arduino Uno, DHT11 temperature sensor, DC motor, transistor, diode, resistor, breadboard, jumper wires, and a USB cable connected to a laptop for power and programming. The Arduino provided the control signal, while the transistor allowed the motor to be powered safely without connecting the motor directly to the Arduino output pin.

4. Circuit Design

The circuit consists of the Arduino microcontroller, which acts as the circuit controller. The DHT11 sensor is attached to a digital input pin to read the temperature. The DC motor is attached through a transistor as it requires more current than the Arduino pins can deliver. The Arduino pin sends a PWM signal to the transistor to control the power delivered to the motor.

Overall, then, the diode is a safety component for the circuit. While it does not play a direct role in controlling the speed of the fan, it helps to protect the Arduino, the laptop, and the rest of the circuit from the voltage spikes that can emerge from the DC motor.

7. Arduino Code Explanation

The Arduino code starts by including the DHT sensor library and defining the sensor pin, sensor type, and motor pin. The DHT11 sensor was connected to pin 2, and the motor control signal was connected to pin 9. Pin 9 was selected because it supports PWM output, which is needed to vary the fan speed.

Inside the loop, the Arduino reads the temperature using `dht.readTemperature()`. If the sensor fails to return a valid reading, the program prints an error message and restarts the loop. If the reading is valid, the temperature is mapped from 23 to 26 degrees Celsius into a fan speed value from 0 to 255. The `constrain()` function keeps the fan speed inside the valid PWM range. Finally, `analogWrite()` sends the PWM signal to the motor pin, and the serial monitor displays the temperature and fan speed.

```
#include "DHT.h"

#define DHTPIN 2
#define DHTTYPE DHT11
#define MOTORPIN 9

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
  pinMode(MOTORPIN, OUTPUT);
}

void loop() {
  float temp = dht.readTemperature();

  if (isnan(temp)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }

  int fanSpeed = map(temp, 23, 26, 0, 255);
  fanSpeed = constrain(fanSpeed, 0, 255);

  analogWrite(MOTORPIN, fanSpeed);

  Serial.print("Temperature: ");
  Serial.print(temp);
  Serial.print(" C, Fan Speed: ");
  Serial.println(fanSpeed);

  delay(2000);
}
```

Arduino code used to read temperature and control fan speed.

8. Testing Process

During the testing of the project, we uploaded the code into the Arduino and used the serial monitor to view the values of both the temperature and fan speed. Each of these parameters was tested to ensure that they were returning values when measured. Additionally, each of the components was tested to ensure the transistor and diode were correctly connected to allow the motor to run while avoiding risking the Arduino pin.

9. Results and Observations

The system was able to read the temperature and change the fan speed based on the code. When the temperature was closer to the lower end of the selected range, the fan speed was lower. When the temperature increased toward the higher end of the range, the PWM value increased and the fan spun faster. This showed that the Arduino was successfully using the sensor reading to control the motor automatically.

Basic system behavior:

Temperature Condition	Arduino Response	Fan Behavior
Low temperature	Small PWM value	Fan spins slowly or stays off
Medium temperature	Moderate PWM value	Fan spins at medium speed
Higher temperature	Large PWM value	Fan spins faster

10. Limitations and Improvements

One of the limitations of this project is that the DHT11 sensor is of a basic nature and is not as accurate or fast responding as more advanced sensors. The temperature range that was chosen for the code was from 23 to 26 degrees Celsius. If the room temperature is outside of this range, the fan will either be running at a very low speed or at full speed. Some improvements that could be made to this project would be to incorporate an LCD screen to display the temperature, use a stronger motor driver, provide an external power supply for the fan, or even change the code to allow the user to select the desired temperature range.

11. Conclusion

Overall, our temperature-controlled fan system successfully demonstrated how a sensor can be used with an Arduino to control an output device. The DHT11 sensor measured temperature, the Arduino processed the reading, and PWM was used to control the speed of the DC motor. The transistor allowed the motor to be switched safely, and the diode protected the circuit from back EMF. This project showed the basic idea behind automatic cooling systems and provided practice with sensor input, motor control, circuit protection, and Arduino programming.